# Traffic Counting and Vehicle Categorization using Computer Vision

# Project Report

Challenge: Proof-of-concept computer vision system for traffic analysis using pre-recorded traffic/CCTV video

Date: January 2025

## 1. Core Solution Concept

The system is a modular pipeline that takes a pre-recorded traffic or CCTV video as input and produces automated vehicle counts, category-wise distribution, and traffic density over time.

**Deep learning–based detection:** A YOLO model identifies and classifies vehicles (cars, motorcycles, buses, trucks) in each frame. **Object tracking:** ByteTrack assigns a unique ID to each vehicle to avoid duplicate counts. **Virtual counting line:** A vehicle is counted only once when its centroid crosses a user-defined line. **Outputs:** Total count, category-wise distribution, traffic density over time, annotated video with bounding boxes and IDs, visual analytics (graphs/dashboards), and a downloadable summary report. The solution is modular and suitable for extension to real-time CCTV feeds.

## 2. Unique Value Proposition

• **One count per vehicle:** Tracking + virtual line ensures each vehicle is counted exactly once.
• **Stable category:** Majority-vote category over the track lifetime reduces misclassification.
• **User-configurable line and thresholds:** Line position (Y), confidence, and direction (up/down/both) are configurable.
• **Dual interfaces:** Desktop GUI (CustomTkinter, tabbed) and optional web interface (Flask).
• **Consistent analytics:** All outputs use the same counted events so numbers add up.
• **Extensible design:** Same pipeline can be extended to live CCTV (RTSP/HTTP stream).

## 3. Key Technologies and Data Used

### 3.1 Technologies

| Technology | Purpose |
|---|---|
| Python 3.8+ | Implementation language |
| YOLO (Ultralytics) | Deep learning object detection; vehicle classification |
| ByteTrack | Multi-object tracking; unique IDs across frames |
| OpenCV (cv2) | Video I/O, drawing bounding boxes and counting line |
| CustomTkinter | Desktop GUI (tabs, cards, modern theme) |

| | |
|---|---|
| Flask | Web server for interactive web dashboard |
| Pandas / NumPy | Aggregation, time-window statistics |
| Matplotlib | Visual analytics (category, density, over time) |
| Jinja2 | HTML report templating |

### 3.2 Data

Input: Pre-recorded traffic or CCTV video (MP4, AVI) from publicly available sources. Pre-trained YOLO (e.g. COCO) with vehicle classes (car, motorcycle, bus, truck, bicycle). No custom training data required. Output: Crossing events (track_id, category, timestamp), aggregated counts, category distribution, time-window counts.

## 4. Methodology

**Detection:** YOLOv8 (nano/small/medium) pre-trained on COCO; vehicle classes only; outputs bounding box, class, confidence, track_id.
**Tracking:** ByteTrack assigns and maintains unique ID per vehicle across frames.
**Virtual counting line:** Line defined by two points (e.g. full width at Y). Centroid of each vehicle tested; crossing = centroid moves from one side to the other in configured direction. Each track_id counted at most once.
**Category:** Per-track histogram; at crossing, category = majority vote. Names normalized (e.g. bus/Bus → Bus).
**Analytics:** Total = crossing events; category distribution = events per category; density over time = events per time window (e.g. 60 s).

## 5. Documentation of Dashboard Functionalities and Analytical Outputs

### 5.1 Desktop Dashboard (CustomTkinter)

Setup: Select video, line position (Y), confidence, options (annotated video, frame limit). Next → Run. Run: Start/Stop, progress bar, status. Results: Total (detected vs counted), category breakdown (counted). Next → Output. Output: Open report, open folder, play annotated video.

### 5.2 Web Dashboard (Flask)

Upload video; set line Y, confidence, frame limit; run processing; view total and category breakdown; view report and annotated video.

### 5.3 Analytical Outputs

Total vehicle count; category-wise distribution (Car, Bus, Truck, etc.); traffic density over time; category over time; annotated video (boxes, IDs, line); downloadable HTML report (summary, tables, charts, sample events). Optional PDF report.

## 6. Policy and Architectural Recommendations

### 6.1 Policy

Privacy and compliance: Use only videos with appropriate rights; for CCTV, comply with local data protection and surveillance laws. Transparency: Document what is counted and how. Human oversight: Treat automated counts as decision support where policy requires.

### 6.2 Architecture

Modularity: Keep detection, tracking, counting, analytics, UI separate. Configuration: Central config for model, line, confidence, paths. Extensibility: Design video source as abstraction (file vs stream) for real-time CCTV. Scalability: GPU server, database for events, REST + web for dashboards.

## 7. Interactive Dashboard / Source Code / URL (if applicable)

Source code: Full Python project (modular). Desktop: python gui.py. Web: python web_gui.py → http://localhost:5000. Repository / URL: (Add your GitHub/GitLab or deployed demo URL here.)

## 8. Final Presentation (PPT / Short Demo Video)

PPT: Title, core concept, value proposition, technologies, methodology, dashboard outputs, policy/architecture, demo, conclusion. Demo video: 3–5 min screen recording (select video, set line, run, view results, report, annotated video, charts). (Add link to PPT or demo video here.)

## References

Ultralytics YOLO: https://github.com/ultralytics/ultralytics

ByteTrack: Multi-Object Tracking by Associating Every Detection Box

OpenCV: https://opencv.org/

CustomTkinter: https://github.com/TomSchimansky/CustomTkinter

End of Project Report